

Title: FPGA Implementation of OTFS Modulation for 6G Waveform Evaluation

1. Introduction & Motivation

Orthogonal Time–Frequency Space (OTFS) modulation is a promising 6G waveform designed for reliable communication in high-mobility and high-Doppler environments. My academic background in digital systems, signal processing, and embedded electronics positions me to contribute to an efficient FPGA-based baseband implementation of OTFS. This project aims to bridge algorithmic understanding with hardware-oriented design principles, enabling real-time and practical evaluation.

2. Why OTFS Modulation for B5G/6G Systems

Traditional **OFDM** (Orthogonal Frequency Division Multiplexing) degrades under high Doppler and fast time-varying channels expected in B5G/6G systems. **OTFS** operates in the delay–Doppler domain, where channel dynamics become sparse and quasi-static, enabling superior robustness in high-mobility environments. Unlike **FDSS** (Frequency-Domain Spectral Shaping), which mainly targets **PAPR** reduction, OTFS fundamentally addresses channel variation and is therefore chosen as the primary waveform for FPGA implementation, with FDSS as a possible future extension.

3. Technical Approach

3.1 Workflow Overview

The implementation follows a structured flow:

Algorithm simulation → **fixed-point validation** → **hardware modularization** → **FPGA testing**, with clear verification checkpoints at each stage.

3.2 Algorithm Simulation (MATLAB / Python)

- Develop a reference OTFS transmitter–receiver chain.
- Implement key processing blocks:
 - ISFFT (Inverse Symplectic Fast Fourier Transform)
 - Delay–Doppler mapping & Windowing
 - FFT / IFFT (Fast Fourier Transform / Inverse FFT)
- Validate bit-error performance under representative channel conditions.
- Convert floating-point operations to fixed-point formats suitable for hardware implementation.

3.3 Hardware Architecture (Verilog / HLS-C)

Hardware modules will be primarily described in **Verilog**, with **High-Level Synthesis (HLS)** explored where C-based descriptions assist rapid prototyping.

Planned FPGA modules include:

- **2D-FFT / 2D-IFFT block:** Streaming, pipelined FFT architecture.
- **Delay–Doppler mapper:** Address-mapped memory with simple arithmetic logic.
- **Windowing FIR module:** Symmetric coefficients to reduce resource utilization.
- **Fixed-point processing:** Consistent Q-format representation across modules.

3.4 Embedded DSP Optimization Strategy

- Streamed dataflow to minimize full-frame buffering.
- Fixed-point arithmetic to reduce FPGA resource usage.
- Pipelined FFT and filtering stages to improve throughput.
- Incremental integration with block-level testbench verification.

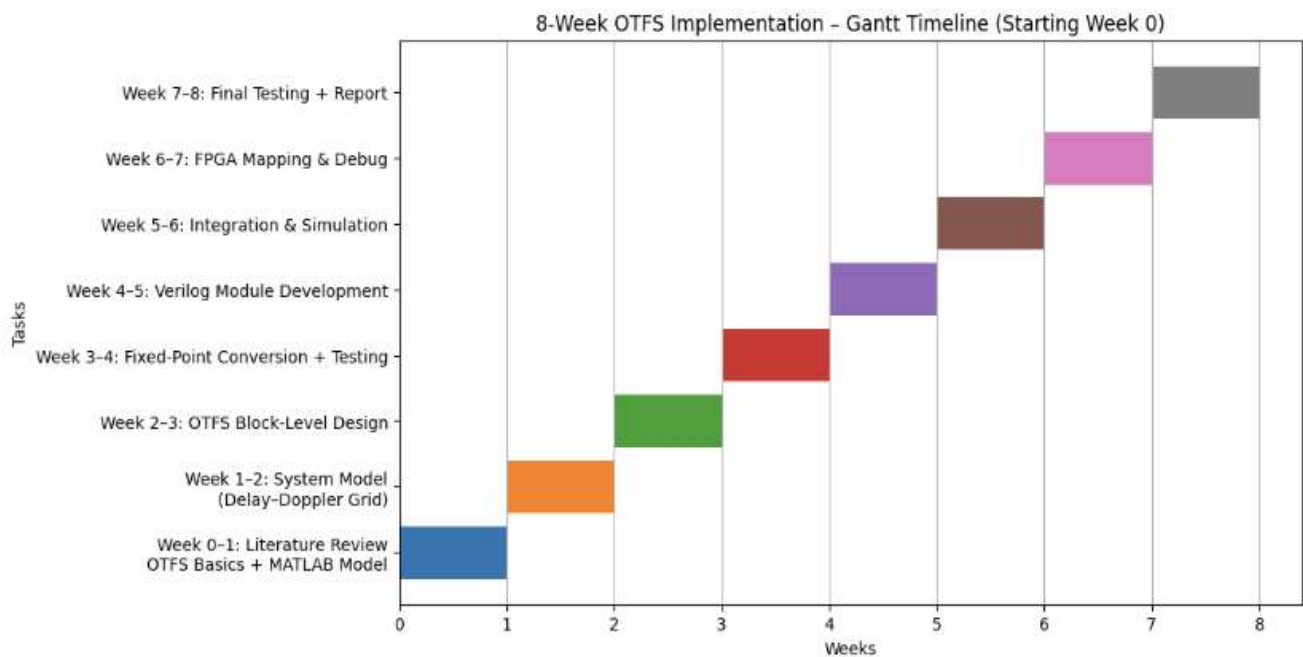
3.5 Verification & Testing

- **MATLAB/Python:** Bit-true validation against hardware results.
- **Simulation:** Block-level verification using ModelSim/Verilator testbenches.
- **FPGA testing:** Top-level integration with loopback testing on an FPGA or SDR evaluation platform.

4. Proposed Timeline (8 Weeks)

The overall development schedule is illustrated in the Gantt chart below.

Figure 1: 8-Week OTFS Implementation Timeline (Gantt Chart)



The timeline progresses from algorithm understanding and simulation to hardware implementation, integration, and final validation.

Table 1: 8-Week OTFS Implementation Timeline

Week	Tasks
Week 0–1	Literature review of OTFS fundamentals; develop MATLAB/Python reference model for modulation–demodulation chain.
Week 1–2	Build a system-level model in the delay–Doppler domain; validate signal flow and grid mapping.
Week 2–3	Block-level OTFS design: FFT/IFFT structure, frame formatting, and interface definition.
Week 3–4	Fixed-point conversion and bit-true verification; validate numerical accuracy against the floating-point model.
Week 4–5	Implement core OTFS modules in Verilog (and/or High-Level Synthesis); develop testbenches.
Week 5–6	Integrate modules into the top-level pipeline; perform functional simulation and initial synthesis.
Week 6–7	FPGA mapping, timing analysis, pipeline optimization, and debugging.
Week 7–8	Final FPGA validation, performance evaluation, result comparison, and report preparation.

5. Availability & Internship Duration

I am available to participate in an 8-week internship from **May 18, 2026, to July 13, 2026**, which follows the guidelines at CCU.

6. Closing Note

I am extremely motivated to work under your guidance on OTFS implementation and meaningfully contribute toward FPGA-based evaluation of 6G waveforms. The project aligns very strongly with my academic interests in the fields of embedded digital signal processing, digital systems, and hardware–algorithm co-design.